

NAME: KARUNYA M A

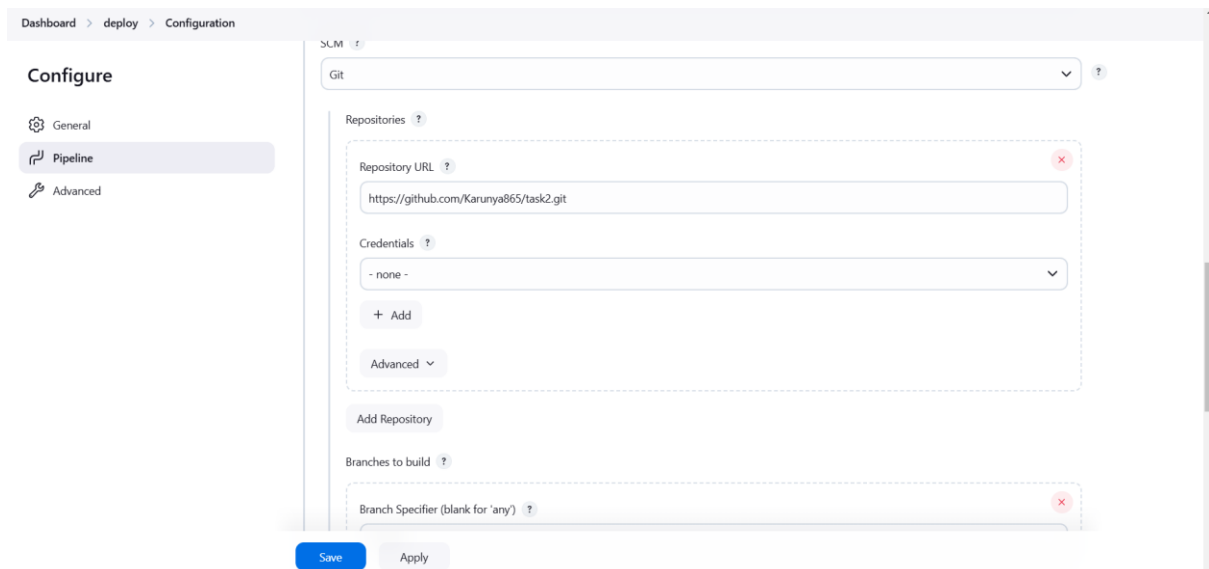
ROLLNO: 22ISR016

JENKINS DEPLOYMENT

This guide explores configuring and managing CI/CD pipelines using Jenkins and Docker. It covers key components like triggers, pipelines, Dockerfiles, and deployment scripts to help enhance your workflow and optimize project implementation.

Pipeline:

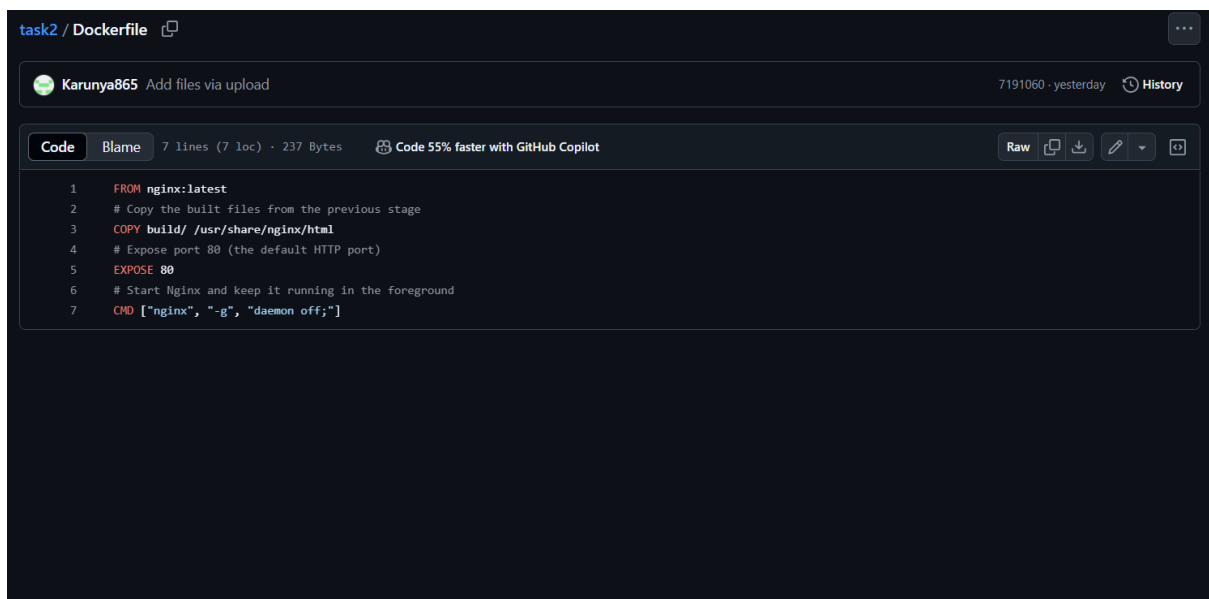
- Defines the pipeline using Groovy or by pulling it from source control.
- **Definition:** Uses a Pipeline script from SCM (Source Control Management).
- **SCM:** Specifies Git as the source code repository.
- **Repository URL:** Points to your GitHub repository.
- **Branches to build:** Targets the 'main' branch for building.
- **Script Path:** Points to the Jenkinsfile located in the root directory of the repository.
- **Additional Behaviours:** Settings related to repository handling and build process.



The screenshot shows the Jenkins 'Configure' page for a pipeline named 'deploy'. The breadcrumb trail at the top is 'Dashboard > deploy > Configuration'. On the left, the 'Configure' section has three tabs: 'General', 'Pipeline' (which is selected), and 'Advanced'. The main content area is titled 'SCM' and shows a dropdown menu set to 'Git'. Below this, there is a 'Repositories' section with a 'Repository URL' field containing 'https://github.com/Karunya865/task2.git'. The 'Credentials' dropdown is set to '- none -'. There is an '+ Add' button and an 'Advanced' dropdown. Below the repository configuration is an 'Add Repository' button. The 'Branches to build' section has a 'Branch Specifier (blank for \'any\')' field. At the bottom, there are 'Save' and 'Apply' buttons.

Dockerfile:

- Defines the Docker container configuration for Nginx.
- Steps include copying built files, exposing port 80, and running Nginx in the foreground.

A screenshot of a code editor showing a Dockerfile. The editor has a dark theme. At the top, there's a header bar with 'task2 / Dockerfile' and a copy icon. Below that, a user profile 'Karunya865' is shown with a green avatar and the text 'Add files via upload'. To the right of the profile, it says '7191060 - yesterday' and 'History'. The main area shows the Dockerfile content with line numbers 1 through 7. The code is as follows:

```
1 FROM nginx:latest
2 # Copy the built files from the previous stage
3 COPY build/ /usr/share/nginx/html
4 # Expose port 80 (the default HTTP port)
5 EXPOSE 80
6 # Start Nginx and keep it running in the foreground
7 CMD ["nginx", "-g", "daemon off;"]
```

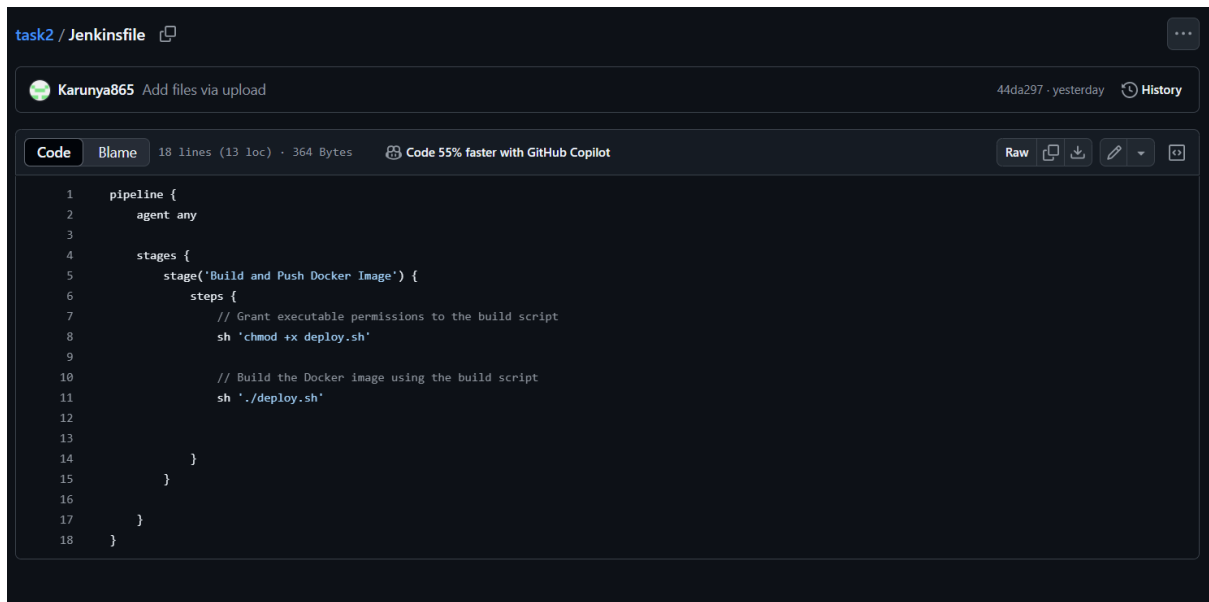
At the bottom of the editor, there are tabs for 'Code' and 'Blame', and a status bar indicating '7 lines (7 loc) · 237 Bytes'. There's also a link to 'Code 55% faster with GitHub Copilot' and buttons for 'Raw', 'Copy', 'Download', 'Edit', and 'Share'.

- ?

?

Jenkinsfile:

- Defines the Jenkins pipeline stages.
- **Build and Push Docker Image** stage: Grants permissions, builds the Docker image, and deploys it.
- Contains script commands for building and pushing the Docker image.

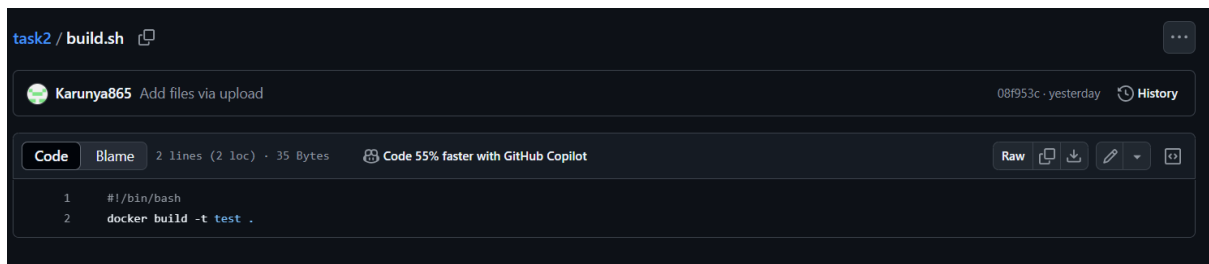


The screenshot shows a GitHub web interface for a file named `Jenkinsfile` in a repository called `task2`. The file is owned by `Karunya865` and was last updated yesterday. The file size is 364 bytes and it contains 18 lines of code. The code is a Jenkins pipeline configuration. It starts with `pipeline {`, followed by `agent any`. Then, there is a `stages {` block containing a `stage('Build and Push Docker Image') {` block. Inside this stage, there are two `steps {` blocks. The first step is `sh 'chmod +x deploy.sh'` with a comment `// Grant executable permissions to the build script`. The second step is `sh './deploy.sh'` with a comment `// Build the Docker image using the build script`. The code ends with `}` for the stage, `}` for the stages block, and `}` for the pipeline.

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Build and Push Docker Image') {
6       steps {
7         // Grant executable permissions to the build script
8         sh 'chmod +x deploy.sh'
9
10        // Build the Docker image using the build script
11        sh './deploy.sh'
12      }
13    }
14  }
15 }
16
17 }
18 }
```

Build.sh:

- Script to build and run the Docker container locally.
- Builds the Docker image and runs the container.

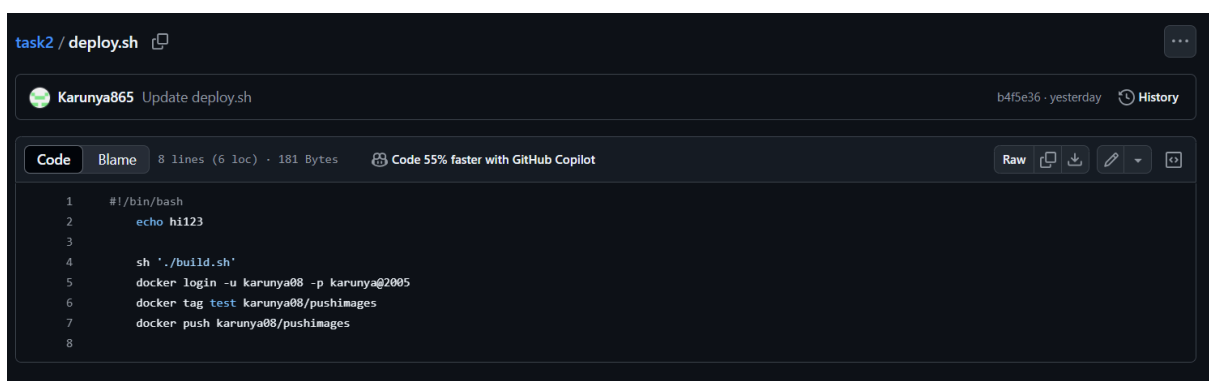


The screenshot shows a GitHub web interface for a file named `build.sh` in a repository called `task2`. The file is owned by `Karunya865` and was last updated yesterday. The file size is 35 bytes and it contains 2 lines of code. The code is a simple shell script that sets the shell to `#!/bin/bash` and runs `docker build -t test .`.

```
1 #!/bin/bash
2 docker build -t test .
```

Deploy.sh:

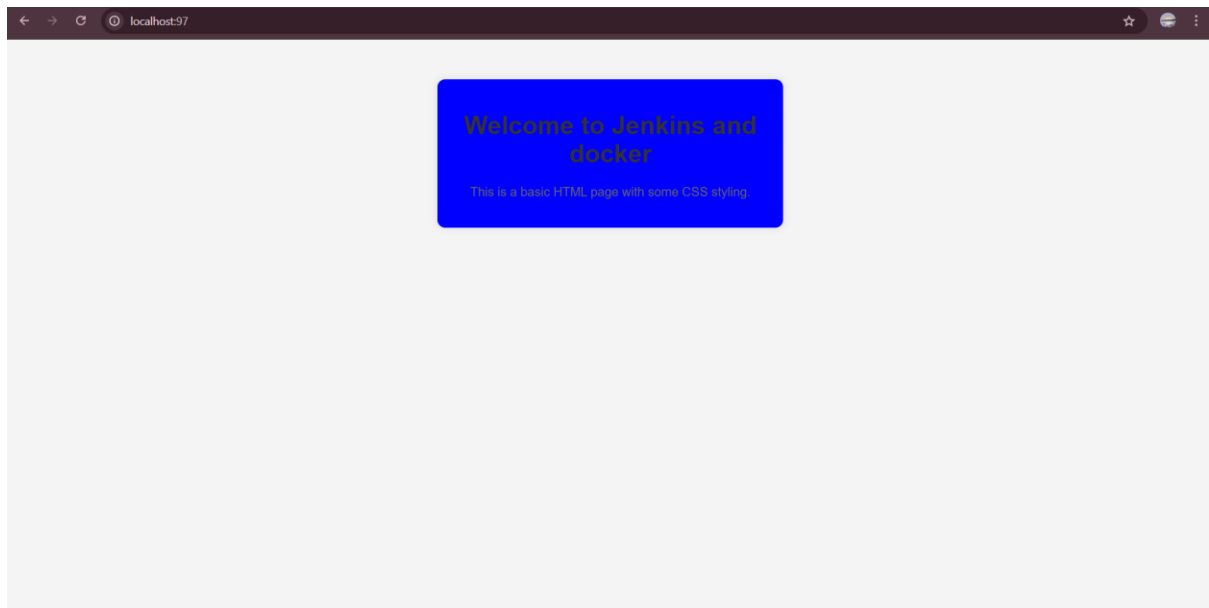
- Script to automate deployment tasks.
- Includes login, tagging, and pushing the Docker image to a repository.



The screenshot shows a GitHub web interface for a file named `deploy.sh` in a repository called `task2`. The file is owned by `Karunya865` and was last updated yesterday. The file size is 181 bytes and it contains 8 lines of code. The code is a shell script that sets the shell to `#!/bin/bash`, prints `hi123`, and then runs three Docker commands: `docker login -u karunya08 -p karunya@2005`, `docker tag test karunya08/pushimages`, and `docker push karunya08/pushimages`.

```
1 #!/bin/bash
2 echo hi123
3
4 sh './build.sh'
5 docker login -u karunya08 -p karunya@2005
6 docker tag test karunya08/pushimages
7 docker push karunya08/pushimages
8
```

OUTPUT



Conclusion:

By understanding and using CI/CD pipelines with Jenkins and Docker, you can revolutionize your development workflow. This guide has equipped you with the knowledge to implement efficient CI/CD strategies, leading to higher productivity and better-quality software.